

TEMA 01

Introducción a Kotlin y Android Studio_

Primeros pasos en el desarrollo de aplicaciones móviles
Android

Configuración

Emuladores

Proyectos

Diseño XML

Eventos de clic

GUÍA DE CONTENIDOS

Índice del tema

Nueve bloques de aprendizaje: desde la configuración de Android Studio hasta el diseño de interfaces y la introducción a los eventos de clic.

-
- 01** **Introducción**
 - 02** **Instalación y configuración**
 - 03** **Dispositivos virtuales y pruebas**
 - 04** **Creación de proyectos**
 - 05** **Estructura del proyecto**
 - 06** **Diseño de interfaz de usuario**
 - 07** **Manejo de eventos de clic**
 - 08** **Ejercicios prácticos**
 - 09** **Recursos adicionales**

Resultados de aprendizaje

Al finalizar este tema podrás instalar el entorno de trabajo, crear una aplicación con vistas XML, ejecutarla en un emulador y programar una respuesta sencilla a un evento de clic.

BLOQUE FORMATIVO

1. Introducción

En esta unidad se presenta el entorno de desarrollo **Android Studio** y el lenguaje de programación **Kotlin**, como punto de partida para el desarrollo de aplicaciones móviles.

Objetivos del tema

- Instalar y configurar Android Studio y las herramientas necesarias para desarrollar aplicaciones Android.
- Reconocer la estructura básica de un proyecto Android y sus archivos principales.
- Explorar las herramientas de diseño de interfaces mediante archivos XML.
- Crear y ejecutar un proyecto en un dispositivo virtual y conocer las pruebas en dispositivos físicos.
- Familiarizarse con los eventos de clic y las acciones asociadas.

Itinerario: instalación → emulador → proyecto → archivos → interfaz → eventos → práctica.

BLOQUE FORMATIVO

2. Instalación y configuración

Android Studio es el entorno de desarrollo con el que crearás y ejecutarás aplicaciones Android. Para trabajar en Windows se necesita una versión de 64 bits compatible; comprueba los requisitos del sistema antes de descargarlo.

2.1. Descargar e iniciar la instalación

1. Accede al [sitio oficial de Android Studio](#) y descarga el instalador correspondiente a tu sistema operativo.
2. Abre el archivo descargado para iniciar el asistente.
3. Revisa los componentes que instalarás y establece la ubicación de los archivos. La ruta predeterminada suele ser suficiente.

2.2. Componentes del entorno

Componente	Función en el curso
Android Studio	Entorno de edición, configuración y ejecución de proyectos Android.
Android SDK	Herramientas y bibliotecas necesarias para el desarrollo de las aplicaciones.
AVD Manager	Herramientas para crear y gestionar dispositivos virtuales.
Virtualización	Permite ejecutar el emulador con aceleración por hardware. En Windows utiliza Windows Hypervisor Platform (WHPX), si tu equipo lo admite.

Antes de empezar: si vas a utilizar el emulador en Windows, comprueba que la virtualización está habilitada y que tu equipo dispone de memoria suficiente. Consulta los requisitos de la versión de Android Studio que vayas a instalar.

2.3. Configuración inicial

1. Deja que finalicen la descarga y extracción de los componentes seleccionados.
2. Elige el tema visual del IDE.
3. Importa ajustes anteriores o inicia una configuración nueva.
4. Descarga el Android SDK y las herramientas que solicite el asistente.
5. Prepara un dispositivo virtual Android si vas a probar las aplicaciones sin conectarlas a un teléfono físico.

BLOQUE FORMATIVO

3. Dispositivos virtuales y pruebas

El **Administrador de dispositivos (Device Manager)** permite crear y gestionar dispositivos virtuales Android (AVD), con los que puedes probar una aplicación sin disponer de varios terminales físicos.

3.1. Crear un dispositivo virtual

1. Abre un proyecto en Android Studio y selecciona **View → Tool Windows → Device Manager**.
2. Pulsa **+** y selecciona *Create Virtual Device*.
3. Elige un modelo de ejemplo o define uno personalizado.
4. Selecciona una imagen de sistema Android para ese dispositivo.
5. Configura las características que necesites: memoria RAM, almacenamiento, orientación, cámara, ubicación y otros sensores.

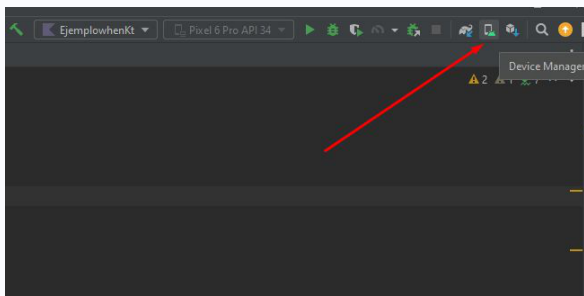


FIGURA · Acceso a la herramienta de dispositivos virtuales

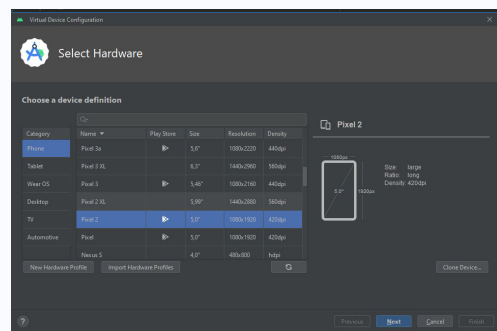


FIGURA · Selección del hardware del dispositivo virtual

Selecciona una imagen de sistema compatible con la arquitectura de tu equipo y la API mínima de la aplicación. Comprueba que la aceleración de hardware está disponible si el emulador funciona con lentitud.

3.2. Iniciar el emulador

1. En el gestor de dispositivos, selecciona el emulador que has creado.
2. Inicialo pulsando el botón de ejecución situado junto al nombre del dispositivo.
3. Espera a que cargue Android y comprueba el funcionamiento de tu aplicación.

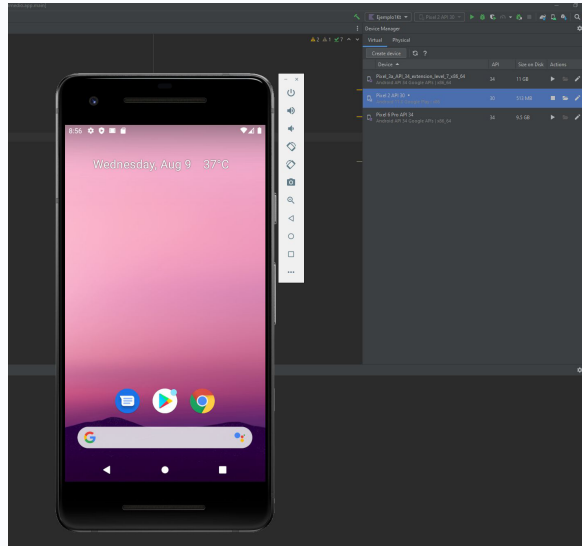


FIGURA · Aplicación en ejecución dentro de un emulador Android

3.3. Seleccionar el dispositivo de ejecución

Antes de ejecutar la aplicación, selecciona el dispositivo en el selector de destino de Android Studio y pulsa **Run**. Para realizar pruebas en un móvil real, conecta el teléfono y activa la depuración USB en sus opciones de desarrollo.

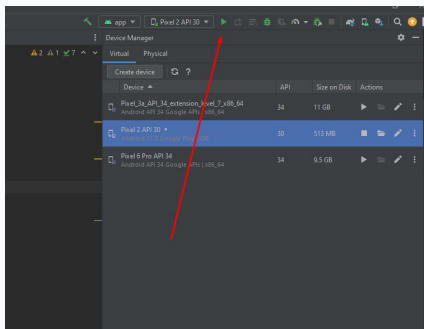


FIGURA · Listado y selección de dispositivos virtuales

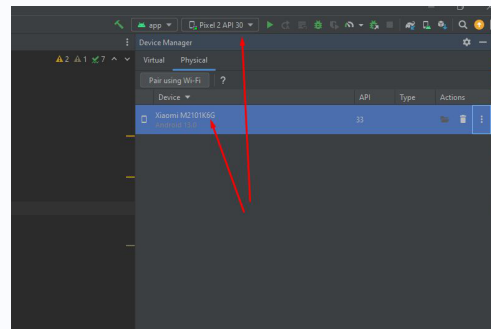


FIGURA · Dispositivo móvil real detectado desde Android Studio

3.4. Emulador en una ventana independiente

El emulador puede mostrarse dentro de Android Studio o en una ventana independiente. Ajusta la opción correspondiente en los ajustes de las herramientas del emulador.

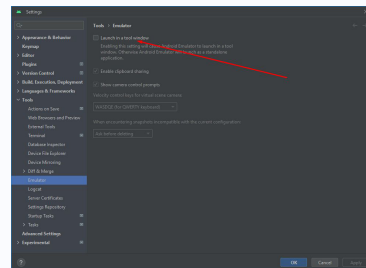


FIGURA · Configuración del emulador en una ventana independiente

BLOQUE FORMATIVO

4. Creación de proyectos

Las **plantillas** proporcionan una estructura inicial para la aplicación. En este tema trabajarás principalmente con **Empty Views Activity** y conocerás la alternativa **No Activity**.

4.1. Plantillas que se utilizarán

Plantilla	Contenido inicial	Uso descrito en el tema
Empty Views Activity	Actividad con una vista inicial vacía.	Diseñar la interfaz y añadir sus componentes.
No Activity	Proyecto sin actividad predefinida.	Crear manualmente las actividades y la interfaz que necesites antes de ejecutar una pantalla.

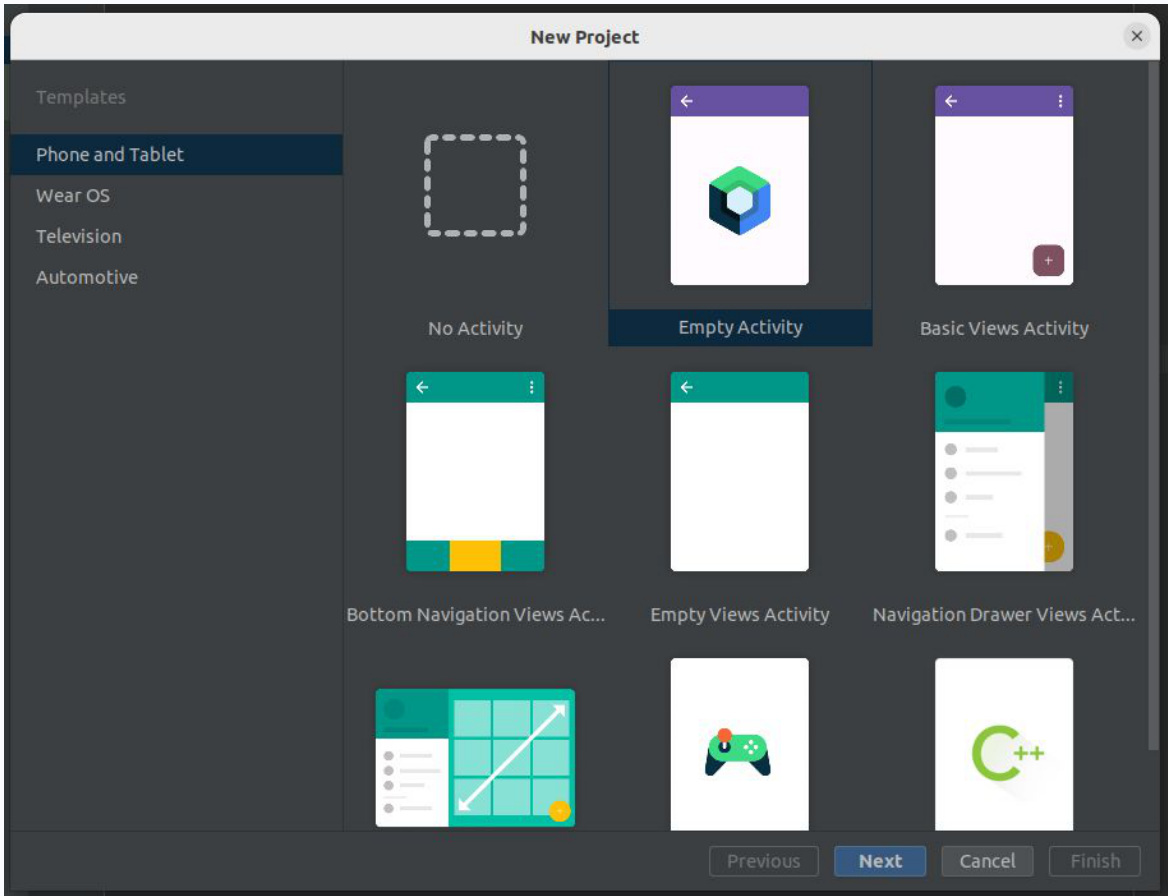


FIGURA · Pantalla de elección de plantilla de Android Studio

4.2. Crear un proyecto paso a paso

1. Abre Android Studio y selecciona la creación de un proyecto nuevo.
2. Elige una plantilla, por ejemplo **Empty Views Activity**.
3. Indica el nombre del proyecto, su ubicación y el lenguaje de programación.
4. Escribe un identificador de paquete con formato inverso, por ejemplo `com.ejemplo.saludo`, y elige una API mínima compatible con el dispositivo donde harás las pruebas.
5. Finaliza el asistente y espera a que se genere la estructura del proyecto.

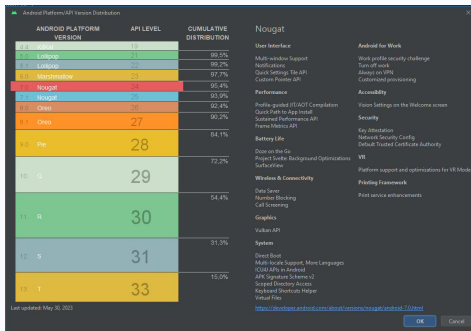


FIGURA · Selección de versión o API de Android

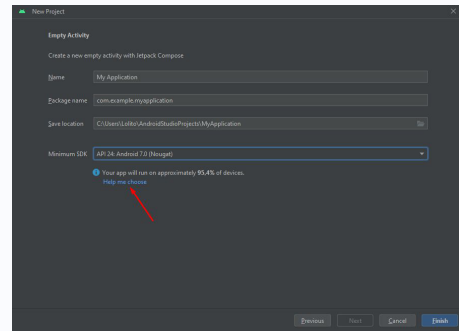


FIGURA · Formulario de configuración del proyecto

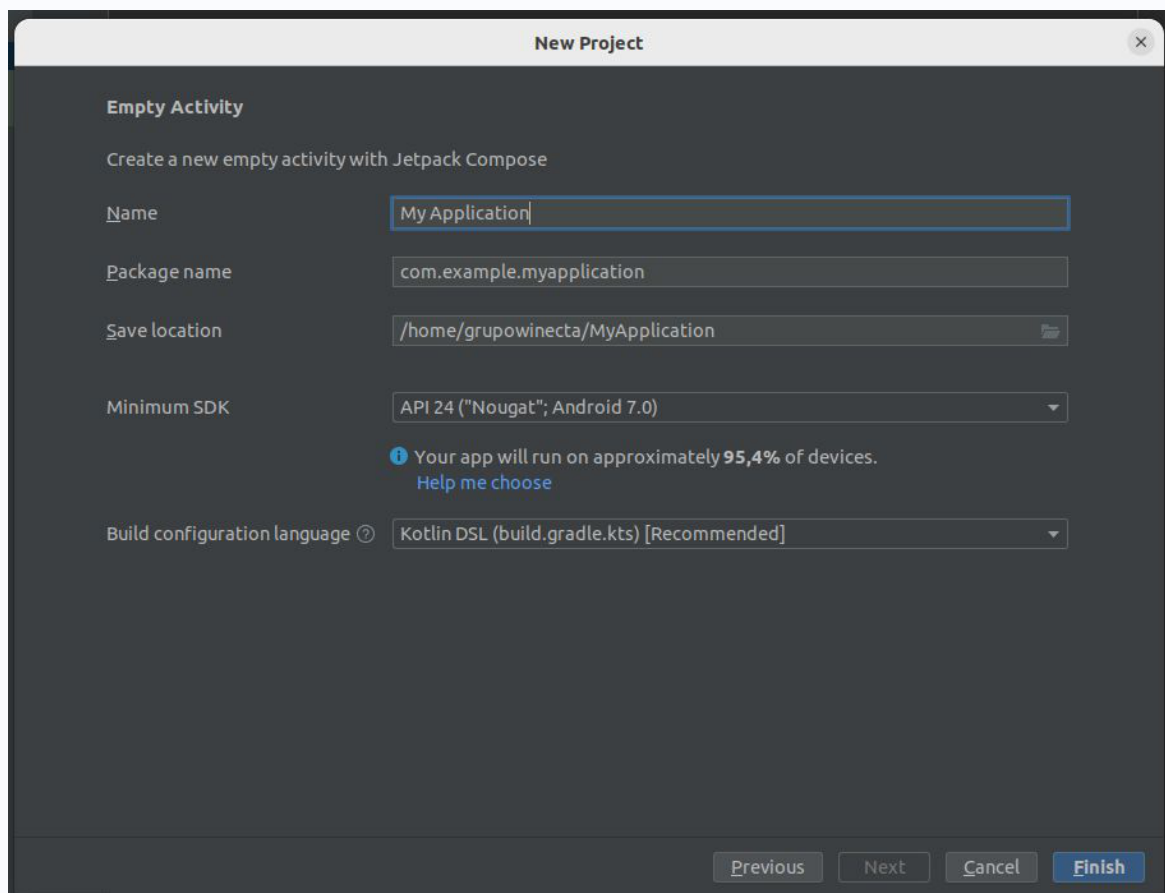


FIGURA · Nombre, paquete, ubicación y API mínima del proyecto

BLOQUE FORMATIVO

5. Estructura de un proyecto Android

Android Studio organiza el código, los recursos y la configuración en carpetas y archivos específicos. Reconocer su función permite localizar los elementos de la aplicación y mantener una estructura de trabajo clara.

5.1. Carpetas principales

Carpeta	Contenido
app/	Módulo de la aplicación; contiene su código, recursos y configuración.
src/	Archivos de código fuente y otros archivos asociados al módulo.
res/	Imágenes, diseños de pantalla y valores reutilizables.
gradle/	Archivos relacionados con la herramienta de construcción Gradle.
build/	Resultados y archivos intermedios generados durante la compilación.

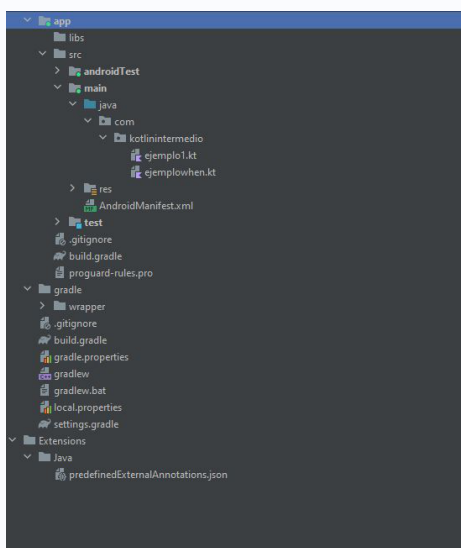


FIGURA · Vista de las carpetas y los archivos de un proyecto

5.2. Archivos de referencia

Archivo	Función
<code>AndroidManifest.xml</code>	Describe la aplicación, sus componentes, permisos y opciones de lanzamiento.
<code>build.gradle</code>	Define aspectos de compilación y dependencias del proyecto.
<code>proguard-rules.pro</code>	Reglas utilizadas para configurar tareas de optimización y protección.
<code>gradle-wrapper.properties</code>	Configuración de la versión de Gradle para el proyecto.
<code>strings.xml</code>	Conjunto de cadenas de texto reutilizables en la aplicación.
<code>activity_main.xml</code>	Archivo de diseño de la interfaz de la actividad principal en el proyecto mostrado.

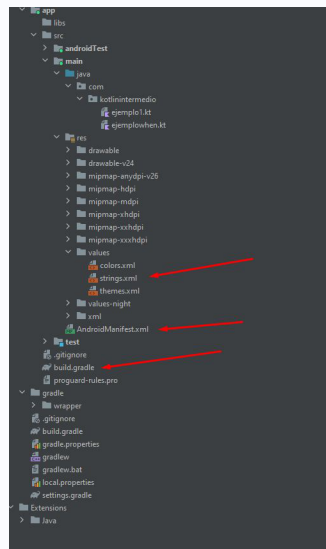


FIGURA · Localización de archivos clave en el árbol del proyecto

5.3. Organización de la carpeta res

- `drawable/` : imágenes y recursos gráficos.
- `layout/` : archivos XML de diseño de interfaz.
- `values/` : cadenas, colores y estilos.
- `mipmap/` : iconos de la aplicación para distintas densidades.

5.4. La función de AndroidManifest.xml

El archivo `AndroidManifest.xml` describe información básica sobre la aplicación, sus actividades y otros componentes, los permisos, la configuración de inicio y metadatos. Android utiliza esta información para reconocer los componentes de la aplicación y su comportamiento.

BLOQUE FORMATIVO

6. Diseño de la interfaz de usuario

En una actividad basada en vistas, el **XML de diseño** describe los componentes de la interfaz. Android Studio permite editar ese XML y previsualizar la pantalla desde el editor visual.

6.1. Abrir el archivo XML de diseño

Localiza un archivo de diseño en la carpeta `res/layout` del proyecto y ábrelo para trabajar con su estructura y su previsualización.

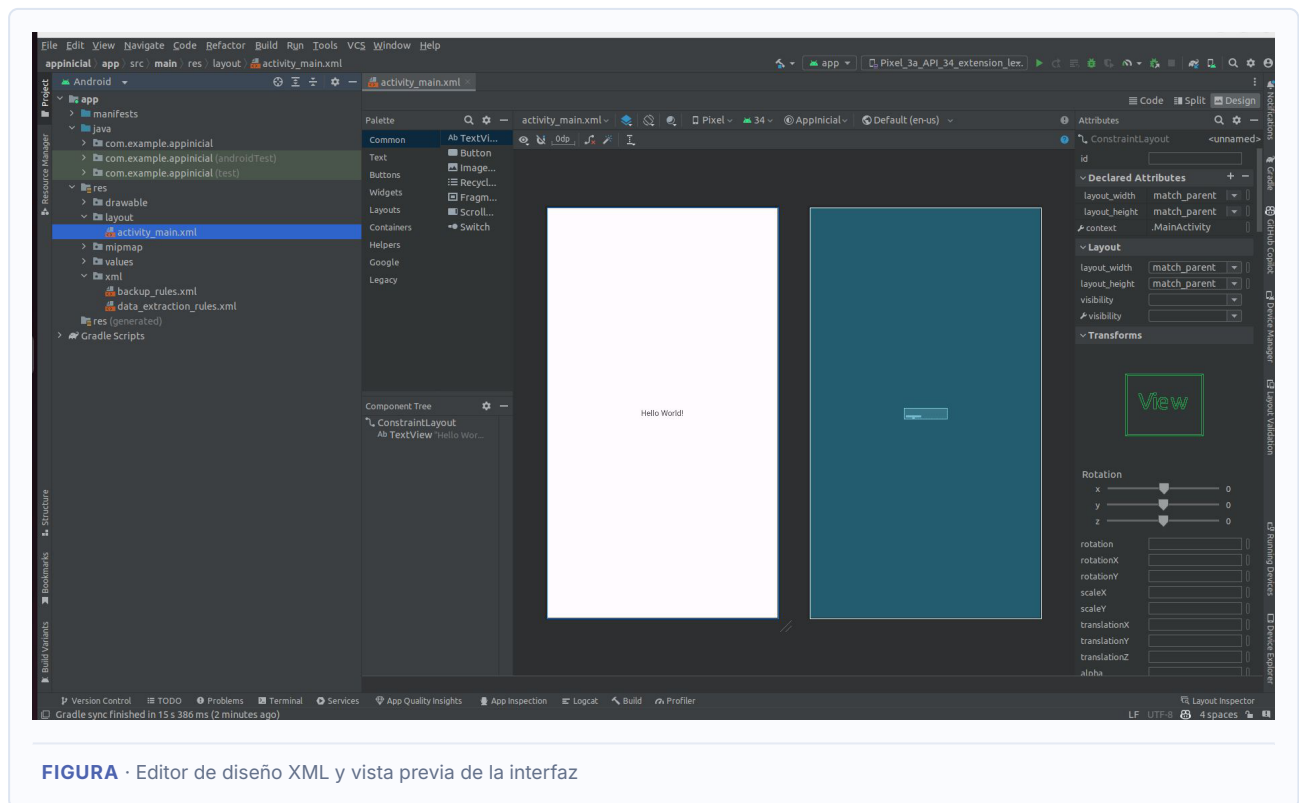


FIGURA · Editor de diseño XML y vista previa de la interfaz

6.2. Añadir elementos a la vista

Desde la paleta de componentes se pueden incorporar elementos visuales, como botones, al diseño de la pantalla. A continuación se observa un botón situado en una vista de ejemplo.

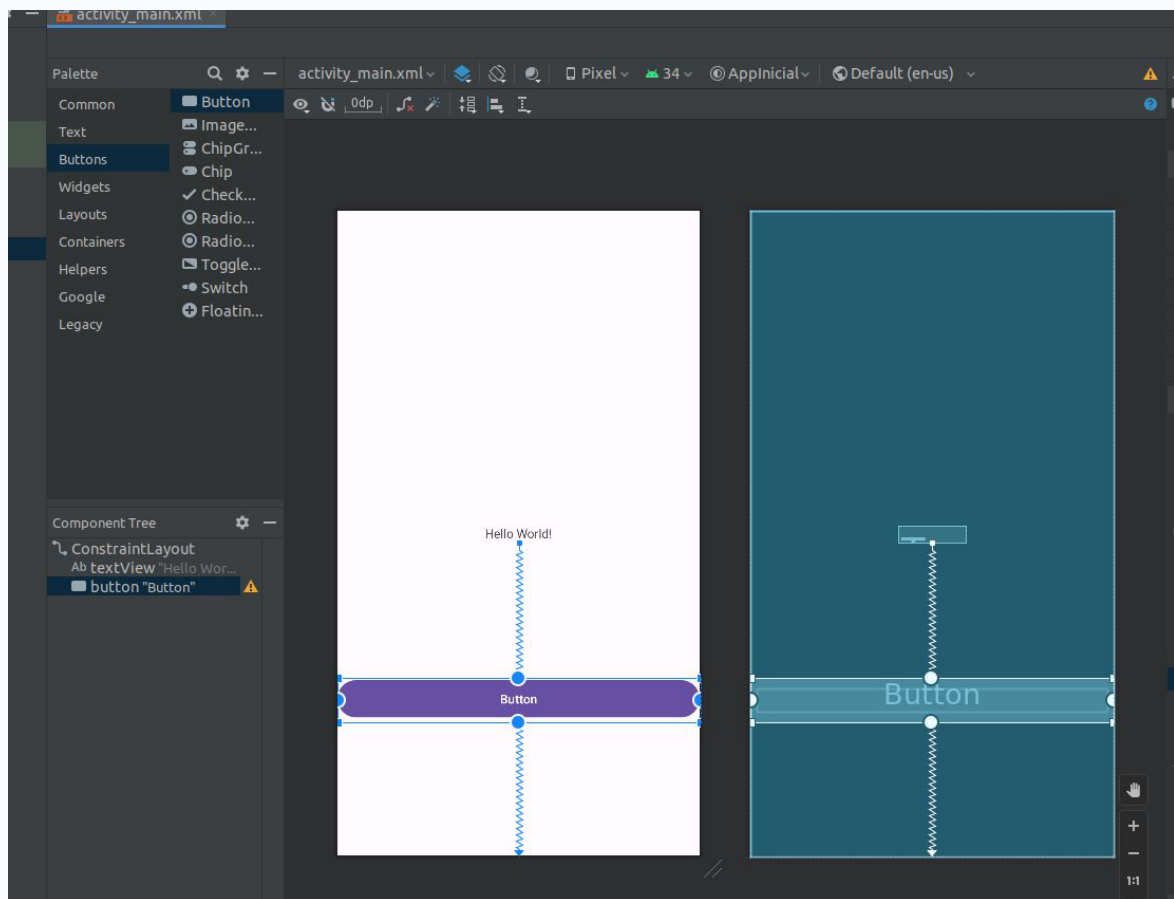


FIGURA · Inserción de un botón mediante el editor visual

6.3. Ajustar posición y tamaño

En un `ConstraintLayout`, define restricciones horizontales y verticales para situar los controles. Utiliza medidas como `wrap_content` para ajustar su tamaño al contenido.

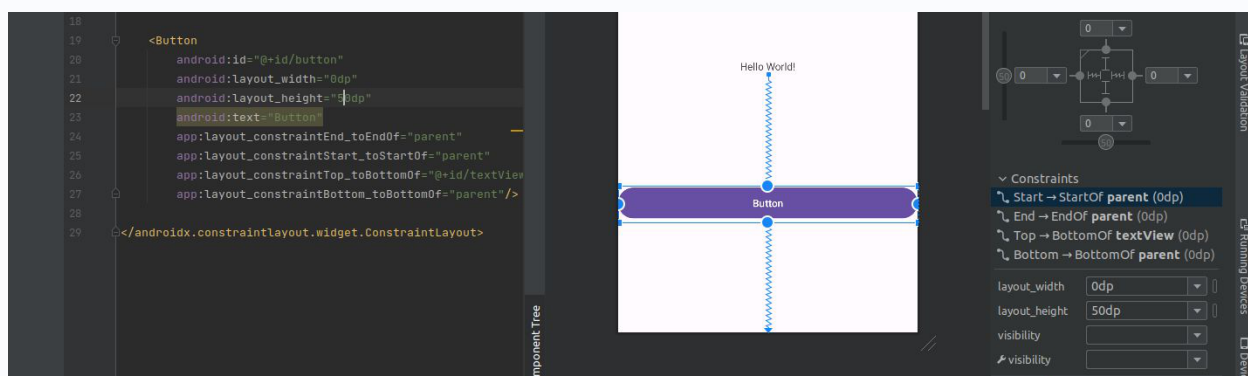
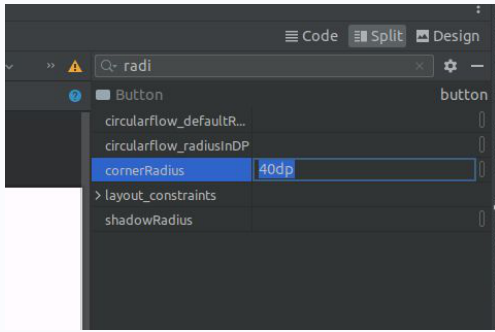
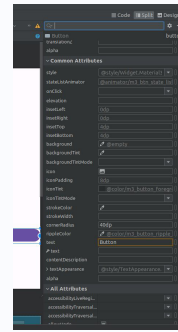


FIGURA · Edición de restricciones y dimensiones de un componente

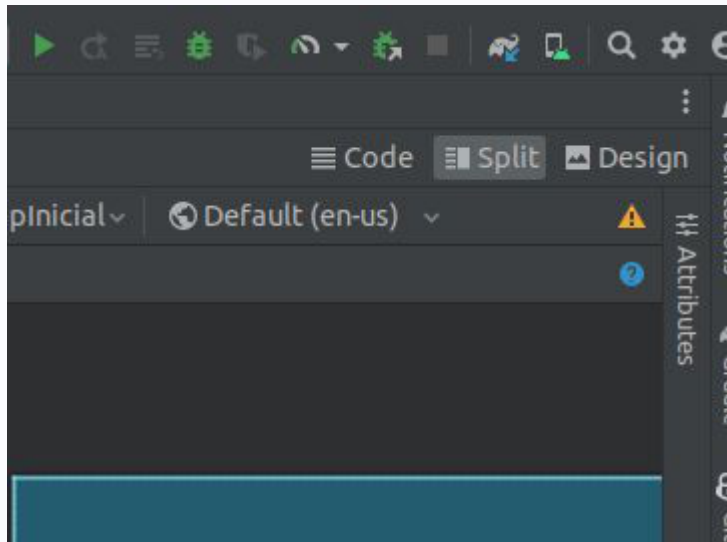
6.4. Personalizar atributos visuales

Selecciona un componente y abre **Atributos** para modificar sus propiedades: identificador, texto, dimensiones o color. Cambia una propiedad y comprueba el resultado en la vista previa.

**FIGURA** · Búsqueda y edición de un atributo visual**FIGURA** · Panel de atributos del componente seleccionado

6.5. Alternar entre Code, Split y Design

El editor ofrece tres modos de trabajo: **Code** (código XML), **Split** (código y diseño) y **Design** (diseño visual).

**FIGURA** · Opciones Code, Split y Design del editor

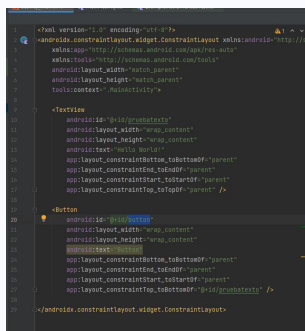
BLOQUE FORMATIVO

7. Manejo de eventos de clic

Un **evento de clic** permite ejecutar una acción cuando el usuario pulsa un botón. Para ello, identifica el componente de la pantalla y asígnale un `setOnClickListener` desde el código Kotlin de la actividad.

7.1. Relación entre interfaz y acción

1. Define o identifica el control en el archivo XML del diseño.
2. Localiza en el código de la actividad el componente que gestionará la interacción.
3. Asocia el evento de clic a la acción que debe ejecutarse.
4. Ejecuta la aplicación y prueba el comportamiento del control.

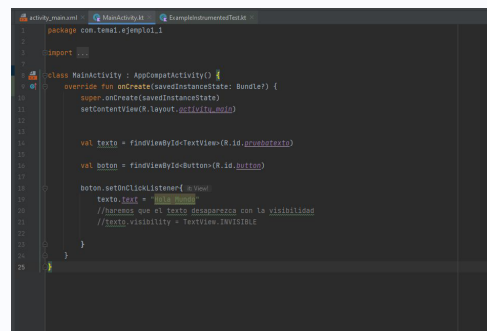


```

1  fun onCreate(savedInstanceState: Bundle?) {
2      super.onCreate(savedInstanceState)
3      setContentView(R.layout.activity_main)
4
5      //TextView
6      val texto = findViewById<TextView>(R.id.textView)
7
8      //Botón
9      val boton = findViewById<Button>(R.id.boton)
10     boton.setOnClickListener {
11         texto.text = "¡Hola Mundo!"
12         //hacemos que el texto desaparezca con la visibilidad
13         //invisible
14         texto.visibility = TextView.INVISIBLE
15     }
16 }

```

FIGURA · Código de la actividad y tratamiento del evento



```

1  <?xml version="1.0" encoding="utf-8"?>
2  <LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
3      android:layout_width="match_parent"
4      android:layout_height="match_parent"
5      android:orientation="vertical"
6      android:gravity="center">
7
8      <TextView
9          android:id="@+id/textView"
10         android:layout_width="wrap_content"
11         android:layout_height="wrap_content"
12         android:text="¡Hola Mundo!"
13         android:layout_marginBottom="10dp"
14         android:layout_constraintTop_toTopOf="parent"
15         android:layout_constraintStart_toStartOf="parent"
16         android:layout_constraintEnd_toEndOf="parent" />
17
18     <Button
19         android:id="@+id/boton"
20         android:layout_width="wrap_content"
21         android:layout_height="wrap_content"
22         android:text="Saludar"
23         android:layout_marginTop="10dp"
24         android:layout_constraintTop_toBottomOf="@id/textView"
25         android:layout_constraintStart_toStartOf="parent"
26         android:layout_constraintEnd_toEndOf="parent" />
27
28 </LinearLayout>

```

FIGURA · Diseño XML con el botón y sus atributos

7.2. Ejemplo completo: botón de saludo

Crea un proyecto con la plantilla **Empty Views Activity**. En `res/layout/activity_main.xml` utiliza el siguiente diseño sencillo con un botón identificado como `botonSaludo`:

XML · activity_main.xml

```

<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:gravity="center">

    <Button
        android:id="@+id/botonSaludo"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Saludar" />

</LinearLayout>

```

En la actividad principal, dentro de `onCreate()`, establece el diseño y configura el clic del botón. Importa `android.os.Bundle`, `android.widget.Button` y `android.widget.Toast` si Android Studio no los añade automáticamente:

KOTLIN · MainActivity.kt · método onCreate()

```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContentView(R.layout.activity_main)

    val boton = findViewById<Button>(R.id.botonSaludo)
    boton.setOnClickListener {
        Toast.makeText(this, "¡Hola desde Kotlin!", Toast.LENGTH_SHORT).show()
    }
}
```

Resultado esperado: al ejecutar la aplicación y pulsar *Saludar*, aparecerá brevemente el mensaje «¡Hola desde Kotlin!».

BLOQUE FORMATIVO

8. Ejercicios prácticos

Realiza estas tres actividades en orden. Guarda el proyecto y comprueba su ejecución en el emulador antes de dar por terminada la práctica.

Actividad 1. Crear un proyecto nuevo

Crea un proyecto Android utilizando una de las plantillas trabajadas. Identifica en el árbol del proyecto el módulo `app` y los archivos básicos de configuración.

Actividad 2. Diseñar una interfaz sencilla

Abre el editor de diseño XML y añade a la pantalla un elemento visual interactivo. Prueba los modos **Code**, **Split** y **Design**, y modifica al menos un atributo del elemento.

Actividad 3. Implementar un evento de clic

Asigna un `setOnClickListener` al botón y muestra un mensaje de confirmación con `Toast`. Ejecuta el proyecto en el emulador y comprueba que el mensaje aparece al pulsarlo.

Secuencia de trabajo: crear proyecto → diseñar pantalla → incorporar clic → ejecutar y comprobar.

BLOQUE FORMATIVO

9. Recursos adicionales

Estos recursos oficiales permiten consultar los procedimientos de instalación, el uso del emulador y los componentes de las aplicaciones Android:

- [Documentación de Android para desarrolladores.](#)
- [Guía de dispositivos virtuales y emulador.](#)
- [Botones y eventos de clic.](#)
- [Instalación de Android Studio.](#)

Para profundizar en el lenguaje de programación, consulta también [la documentación oficial de Kotlin.](#)